Documento de Decisiones de Diseño

UNIVERSIDAD DE MANIZALES

FACULTAD DE CIENCIAS E INGENIERÍAS

Construcción de un Sistema de Gestión Académica utilizando Programación

Actividad Colaborativa N.º 2: Evolución del Sistema de Gestión Académica

Integrantes: Juan Sebastián Jaramillo

Docente: Natalia Marcela Castellanos Gomez

Fecha: 1 de Noviembre de 2026

Repositorio: [GitHub - JuanJaraa/sistema_academico · GitHub]

1. Introducción

Este documento explica y justifica las decisiones de diseño tomadas al evolucionar el Sistema de Gestión Académica de la Actividad Colaborativa N.º 1. La evolución se hizo sobre el mismo proyecto, sin reemplazarlo: se conservaron las clases `Persona`, `Estudiante`, `Docente`, `Administrativo`, `ProgramaAcademico` y `Asignatura`, la estructura Modelo-Vista-Controlador y el recorrido polimórfico, y se agregaron clases abstractas, composición, la oferta de grupos con matrícula y excepciones personalizadas.

Cada cambio aparece marcado en el código con comentarios `[AC2 - NUEVO]` o `[AC2 - MODIFICADO]` y resumido en el archivo `CAMBIOS_AC2.md`.

2. Descripción general de la solución implementada

La solución agrega tres archivos al paquete `models/`: `horario.py` (clase `Horario`), `grupo.py` (clase `GrupoAcademico`) y `excepciones.py` (cinco excepciones personalizadas). Además se modificó `persona.py`, para que `Persona` sea abstracta, y `controlador.py`, para agregar las pruebas de las nuevas funciones. Las demás clases del modelo, la vista y `main.py` no cambiaron.

El modelo sigue sin imprimir nada: los métodos devuelven texto y la `Vista` lo muestra. Lo nuevo sigue esa misma convención, por ejemplo `GrupoAcademico.mostrar_informacion_grupo()` y `mostrar_estudiantes()` devuelven cadenas, y es el `Controlador` quien las envía a `Vista`.

3. Desarrollo

3.1 La clase Persona como clase abstracta

En la Actividad 1, `Persona` se podía instanciar y su método `realizar_actividad_principal()` devolvía un texto genérico ("es una persona de la universidad"). Ese comportamiento no tiene sentido en el dominio: dentro de la universidad toda persona es un estudiante, un docente o un administrativo, y lo que una persona hace como actividad principal solo se puede determinar en la clase especializada.

Por eso `Persona` ahora hereda de `ABC` y `realizar_actividad_principal()` se declaró con `@abstractmethod`, sin implementación. Esto obliga a toda subclase a escribir su propia versión y hace que Python lance un `TypeError` si alguien intenta ejecutar `Persona(...)`. El `Controlador` lo demuestra en `probar_persona_abstracta()`, que captura el `TypeError` y muestra un mensaje en español. El resto de `Persona` (atributos privados, getters, setters y `mostrar_informacion()`) se dejó igual, porque sigue siendo el comportamiento común que heredan las tres clases hijas.

3.2 Implementación del método abstracto en las clases derivadas

`Estudiante`, `Docente` y `Administrativo` ya implementaban `realizar_actividad_principal()` desde la Actividad 1, por lo que no necesitaron modificaciones. Cada una aporta un comportamiento distinto según su responsabilidad:

- `Estudiante` indica el programa que cursa y su semestre actual.
- `Docente` indica que orienta clases en su facultad.
- `Administrativo` indica el cargo que ejerce y la dependencia donde trabaja.

La diferencia con la versión anterior es que ahora esa implementación es obligatoria. Si en el futuro se creara otra clase hija de `Persona` y se olvidara el método, Python impediría instanciarla.

3.3 Relación entre GrupoAcademico, Asignatura, Docente, Estudiante y Horario

Una asignatura puede ofrecerse en varios grupos. Por eso se creó `GrupoAcademico` como una clase independiente que relaciona a los demás objetos:

- **Asignatura:** el constructor de `GrupoAcademico` recibe un objeto `Asignatura` ya creado y lo guarda en `self.__asignatura`. No se guarda solo el código o el nombre como texto.
- **Docente:** igualmente, recibe un objeto `Docente` ya creado como responsable del grupo.

- **Estudiante:** el grupo mantiene una lista privada de objetos `Estudiante` matriculados, con el límite de `capacidad_maxima`. Se opera con `matricular_estudiante(estudiante)`, `retirar_estudiante(codigo_estudiantil)` y `mostrar_estudiantes()`.
- **Horario:** el grupo crea y contiene un objeto `Horario` (ver sección 3.4).

En la prueba del controlador se crearon tres grupos: `G01` y `G02` ofrecen la misma asignatura (Programación Orientada a Objetos) con docentes y horarios distintos, y `G03` ofrece Cálculo Diferencial. Así se evidencia que una asignatura puede tener varios grupos.

Se usó la asociación por objetos (y no por textos) porque permite que el grupo consulte los datos reales de sus participantes, por ejemplo `self.__docente.get_nombre()` al mostrar la información del grupo, y evita que el nombre de un docente quede duplicado y desactualizado en varios lugares.

3.4 Composición: por qué el grupo contiene un horario y no hereda de él

`Horario` guarda día, hora de inicio, hora de fin y salón. Dentro del constructor de `GrupoAcademico` se ejecuta `self.__horario = Horario(dia, hora_inicio, hora_fin, salon)`. Es decir, el grupo crea su propio horario como parte de su organización, y ese horario no se comparte ni existe por separado: describe cuándo y dónde se desarrolla ese grupo en particular. Esta es una relación de **composición** ("un grupo tiene un horario").

No corresponde a herencia porque la herencia expresa una relación "es un", y un grupo académico no es un tipo de horario. El grupo organiza la oferta (asignatura, docente, cupo y estudiantes), mientras que el horario solo describe tiempo y lugar. Si `GrupoAcademico` heredara de `Horario`, el grupo "sería" un horario y no podría, por ejemplo, tener un horario como atributo propio. Además, el horario no se representó como una sola cadena de texto sino como un objeto, lo que permite consultar el día, las horas y el salón por separado (`get_dia()`, `get_hora_inicio()`, `get_hora_fin()`, `get_salon()`).

Una decisión que se tomó aquí: el `Horario` se construye dentro del grupo (recibiendo sus cuatro datos) en lugar de recibirse ya armado. Se hizo así para reflejar de forma más estricta la composición, porque el ciclo de vida del horario depende del grupo.

3.5 Excepciones personalizadas

Todas las excepciones se definieron en `models/excepciones.py` y heredan de una clase base propia, `SistemaAcademicoError`, que a su vez hereda de `Exception`. Esto permite capturar cualquiera de ellas de forma general si se necesita, o individualmente como se hace en el controlador. Cada excepción construye un mensaje comprensible con los datos del caso (códigos de estudiante, grupo, asignatura o programa).

Excepción	Situación que controla	Dónde se lanza (`raise`)
`EstudianteYaMatriculadoError`	El estudiante ya está en el grupo	`GrupoAcademico.matricular_estudiante()`
`GrupoLlenoError`	El grupo llegó a su capacidad máxima	`GrupoAcademico.matricular_estudiante()`
`EstudianteNoMatriculadoError`	Se intenta retirar a un estudiante que no está en el grupo	`GrupoAcademico.retirar_estudiante()`
`AsignaturaNoEncontradaError`	La asignatura buscada no existe	`Controlador.buscar_asignatura()`
`ProgramaNoEncontradoError`	El programa buscado no existe	`Controlador.buscar_programa()`

En `matricular_estudiante()` primero se verifica si el estudiante ya está matriculado (comparando por `get_codigo()`) y después si hay cupo, de modo que un estudiante duplicado reciba el error que realmente le corresponde aunque el grupo esté lleno.

Las excepciones se **lanzan** en el método donde se detecta la situación y se **capturan** en el `Controlador` (métodos `probar_matriculas()`, `probar_retiros()` y `probar_busquedas()`), que es lo que ejecuta `main.py`. Allí se muestra el mensaje con `Vista.error()` y el programa continúa con la siguiente prueba. Se respetó la separación del patrón MVC: el modelo detecta y lanza, el controlador decide qué hacer y la vista muestra. No se usaron condicionales que solo impriman mensajes.

Las dos búsquedas se implementaron en el controlador porque es allí donde viven las listas `self.programas` y `self.asignaturas`.

3.5.1 Pruebas realizadas

Al ejecutar `python main.py`, después de las secciones 1 a 7 de la Actividad 1, el programa demuestra: el intento de crear una `Persona` (falla), la creación de asignaturas, docentes, estudiantes y horarios, tres grupos, matrículas correctas, un intento de matrícula duplicada, un intento de superar la capacidad del grupo `G01` (capacidad 2), un retiro correcto, un retiro de un estudiante inexistente y la búsqueda de una asignatura y un programa que no existen. Tras cada excepción se muestra "El programa continúa su ejecución" y la ejecución sigue hasta el final.

3.6 Cómo se mantiene el polimorfismo de la Actividad 1

El recorrido polimórfico de la Actividad 1 (`recorrido_polimorfico()` en el controlador) no se modificó: un solo ciclo `for` recorre la lista `self.personas` y llama a `mostrar_informacion()` y `realizar_actividad_principal()` sin preguntar de qué clase es cada objeto. Python ejecuta la versión correspondiente a `Estudiante`, `Docente` o `Administrativo`.

La clase abstracta incluso refuerza este polimorfismo: como `realizar_actividad_principal()` es abstracto, ninguna instancia de `Persona` puede existir sin esa implementación, así que cualquier objeto de la lista está garantizado de responder al método. Además, los estudiantes que se matriculan en un grupo siguen siendo objetos `Estudiante` (hijos de `Persona`), y el nuevo estudiante creado durante las pruebas de matrícula se agregó a la misma lista `self.personas`, por lo que también participa en el diseño polimórfico. El polimorfismo se aplica así a la parte nueva sin duplicar código.

4. Conclusiones

- Convertir `Persona` en clase abstracta hizo que el modelo reflejara mejor la realidad de la universidad, sin cambiar las clases hijas, que ya cumplían el contrato.
- La composición permitió representar el horario como parte del grupo mediante objetos y no como texto, y deja clara la diferencia entre una relación "tiene un" y una de herencia "es un".
- Las excepciones personalizadas separan la detección de un problema (en el modelo) de su manejo (en el controlador) y permiten que el programa siga ejecutándose con mensajes claros.
- Evolucionar el proyecto marcando cada cambio facilitó conservar la Actividad 1 intacta, tal como pide el enunciado.
- Como mejora futura, se podría validar que la hora de fin sea posterior a la de inicio y que dos grupos del mismo docente no se crucen en el horario.

5. Declaración de uso de herramientas de inteligencia artificial

Se utilizó Claude (Anthropic) como apoyo para comprender los requerimientos del enunciado, proponer la estructura de las nuevas clases (`Horario`, `GrupoAcademico` y las excepciones), generar fragmentos de código de la Actividad 2 y redactar un borrador de este documento y de la actualización del README. El equipo revisó el código, lo ejecutó y

es responsable de comprender y poder explicar y justificar la totalidad del trabajo entregado. *(Los integrantes deben revisar este documento y ajustarlo con sus propias palabras y con los datos reales de participación de cada uno.)*